ESGI — École Supérieure de Génie Informatique

Spécialisation Cybersécurité — 4ème Année

Le Chiffrement Homomorphique

État de l'art, enjeux et perspectives pour la cybersécurité

Papier de Recherche Académique

Auteurs : COUPY Kévin, RUAULT Clément

Date : 24 MARS 2026

Résumé

L'essor du cloud computing et la numérisation massive des données personnelles et professionnelles ont fait émerger un défi fondamental de la cryptographie moderne : peut-on réaliser des calculs sur des données chiffrées sans jamais avoir à les déchiffrer ? Le chiffrement homomorphique (FHE, Fully Homomorphic Encryption) répond affirmativement à cette question en permettant d'effectuer des opérations arithmétiques arbitraires directement sur des données chiffrées, produisant un résultat chiffré dont le déchiffrement donne le résultat attendu du calcul en clair. Ce papier constitue un état de l'art complet du chiffrement homomorphique, destiné aux étudiants en cybersécurité de 4ème année. Nous retraçons l'évolution historique du domaine depuis les travaux précurseurs de Rivest, Adleman et Dertouzos (1978) jusqu'à la percée de Craig Gentry (2009) et aux quatre générations de schémas qui ont suivi : PHE, SHE, et les schémas FHE modernes tels que BGV, BFV, CKKS et TFHE. Nous analysons en profondeur les fondements mathématiques (problème LWE, Ring-LWE), les mécanismes du bootstrapping, et l'écosystème industriel actuel — en particulier Zama et ses bibliothèques open-source Concrete, Concrete ML et TFHE-rs, ainsi que Microsoft SEAL, IBM HElib et OpenFHE. Les applications concrètes dans les domaines de la santé, de la finance, du machine learning confidentiel et de la blockchain sont détaillées. Enfin, nous présentons honnêtement les limites actuelles — surcout computationnel, complexité de mise en œuvre, manque de maturité — ainsi que les perspectives d'accélération matérielle (programme DARPA DPRIVE, puce HERACLES d'Intel) et de convergence avec d'autres paradigmes cryptographiques. Nous concluons que le FHE, bien qu'encore en phase de maturation, est appelé à devenir un pilier incontournable de la cybersécurité de demain.

Abstract

The rise of cloud computing and the massive digitization of personal and professional data have given rise to a fundamental challenge in modern cryptography: can computations be performed on encrypted data without ever decrypting it? Fully Homomorphic Encryption (FHE) answers this question affirmatively by allowing arbitrary arithmetic operations to be performed directly on encrypted data, producing an encrypted result whose decryption yields the expected plaintext computation result. This paper provides a comprehensive state-of-the-art survey of homomorphic encryption, aimed at fourth-year cybersecurity students. We trace

the historical evolution of the field from the pioneering work of Rivest, Adleman and Dertouzos (1978) through Craig Gentry's breakthrough (2009) and the four subsequent generations of schemes: PHE, SHE, and modern FHE schemes such as BGV, BFV, CKKS and TFHE. We analyze in depth the mathematical foundations (LWE problem, Ring-LWE), the bootstrapping mechanism, and the current industrial ecosystem — in particular Zama and its open-source libraries Concrete, Concrete ML and TFHE-rs, as well as Microsoft SEAL, IBM HElib and OpenFHE. Concrete applications in healthcare, finance, confidential machine learning and blockchain are detailed. Finally, we honestly present current limitations — computational overhead, implementation complexity, lack of maturity — as well as the prospects for hardware acceleration (DARPA DPRIVE program, Intel's HERACLES chip) and convergence with other cryptographic paradigms. We conclude that FHE, while still maturing, is set to become an essential pillar of tomorrow's cybersecurity.

Table des matières

1. Introduction 6
2. Fondements théoriques du chiffrement homomorphique 8
  2.1 Rappels cryptographiques 8
  2.2 Taxonomie du chiffrement homomorphique 9
    2.2.1 Chiffrement Partiellement Homomorphique (PHE) 9
    2.2.2 Chiffrement Quelque peu Homomorphique (SHE) 9
    2.2.3 Chiffrement Pleinement Homomorphique (FHE) 10
  2.3 Évolution historique 10
3. Schémas FHE modernes — Analyse technique approfondie 12
  3.1 Le problème LWE (Learning With Errors) 12
  3.2 Schémas BFV et BGV 13
  3.3 Schéma CKKS 14
  3.4 Schéma TFHE 15
  3.5 Le bootstrapping : concept, coût et optimisations 16
4. L'écosystème industriel et open-source 18
  4.1 Zama — Pionnier du FHE accessible 18
    4.1.1 TFHE-rs 18
    4.1.2 Concrete 18
    4.1.3 fhEVM 19
  4.2 Autres acteurs majeurs 19
  4.3 Standardisation 20
5. Applications concrètes et cas d'usage 22
  5.1 Santé et données médicales 22
  5.2 Finance et secteur bancaire 23
  5.3 Machine learning confidentiel 23
  5.4 Cloud computing et souveraineté des données 25
  5.5 Blockchain et Web3 25
6. Limites, défis et critiques 27
  6.1 Performances — L'obstacle principal 27
  6.2 Complexité d'implémentation 27
  6.3 Maturité et confiance 28
  6.4 Menace quantique et sécurité post-quantique 29
7. Perspectives et futur du FHE 31

7.1 Accélération matérielle 31
7.2 Convergence avec d'autres technologies 31
7.3 Démocratisation du FHE 32
8. Conclusion 34
Bibliographie 36
Annexes 39
Annexe A — Glossaire des termes techniques 39
Annexe B — Tableau comparatif des schémas FHE 40
Annexe C — Exemple de code : addition homomorphique avec Concrete ML 41

1. Introduction

Le monde numérique du XXIe siècle est caractérisé par une explosion sans précédent du volume de données générées, collectées et traitées. En 2025, le volume mondial de données atteint plusieurs dizaines de zettaoctets, alimenté par la prolifération des objets connectés (IoT), des réseaux sociaux, des systèmes de santé numérisés et de l'intelligence artificielle. Parallèlement, le cloud computing s'est imposé comme le paradigme dominant du traitement informatique : selon des estimations récentes, plus de 90 % des entreprises du Fortune 500 utilisent des services cloud publics pour héberger tout ou partie de leurs infrastructures critiques[11].

Ce déport massif des données vers des serveurs tiers crée une tension fondamentale avec les exigences de confidentialité. Lorsqu'une organisation délègue le traitement de données sensibles — médicales, financières, stratégiques — à un prestataire cloud, elle lui accorde, dans le modèle cryptographique classique, un accès en clair à ces données le temps du traitement. Même si le prestataire est contractuellement de confiance, cette exposition constitue un vecteur d'attaque critique : compromission du serveur, accès non autorisé par des employés malveillants, injonction légale d'un gouvernement étranger, ou simple erreur de configuration.

Le Règlement Général sur la Protection des Données (RGPD) de l'Union Européenne[12], entré en vigueur en mai 2018, a durci le cadre réglementaire autour de la protection des données personnelles. Il impose aux organisations de mettre en place des mesures techniques appropriées pour garantir la confidentialité, l'intégrité et la disponibilité des données. Dans un contexte de souveraineté numérique de plus en plus prégnant — illustré par les débats autour du Cloud Act américain et les velléités d'autonomie technologique européenne — la question du contrôle des données prend une dimension géopolitique. L'approche zero-trust, paradigme de sécurité qui postule qu'aucun acteur — interne ou externe — ne doit être considéré comme de confiance par défaut, impose de repenser en profondeur les architectures de traitement de données[13].

C'est dans ce contexte que se pose la problématique centrale du présent papier : comment effectuer des calculs sur des données chiffrées sans jamais les déchiffrer ? Cette question, qui peut sembler paradoxale — car le chiffrement a précisément pour vocation de rendre les

données inintelligibles — trouve une réponse dans le concept de chiffrement homomorphique. Un cryptosystème homomorphique permet d'effectuer des opérations algébriques sur des données chiffrées, telles que l'addition ou la multiplication, dont le déchiffrement du résultat produit exactement le résultat de l'opération effectuée sur les données en clair, sans que les données elles-mêmes aient jamais été exposées.

Formellement, un schéma de chiffrement Enc est dit homomorphique pour une opération $\circ$ si, pour tout messages m_1 et m_2 : $\text{Enc}(m_1) \oplus \text{Enc}(m_2) = \text{Enc}(m_1 \circ m_2)$, où $\oplus$ désigne l'opération correspondante dans l'espace des chiffrés. La notion de chiffrement Fully Homomorphic Encryption (FHE), qui supporte un ensemble arbitrairement large d'opérations — équivalent à tout circuit booléen — représente ce que les cryptographes appellent le « Saint Graal » de la cryptographie. Elle permettrait, par exemple, à un hôpital de déléguer l'analyse de dossiers médicaux sensibles à un serveur cloud sans que ce dernier puisse jamais accéder aux données réelles des patients[1].

L'enjeu de cybersécurité est considérable. Dans le contexte actuel de menaces persistantes avancées (APT), de ransomwares ciblant les infrastructures critiques, et de l'avènement des ordinateurs quantiques qui menacent les fondements de la cryptographie asymétrique classique, le FHE offre un modèle de sécurité radicalement nouveau : même un serveur compromis ne peut pas accéder aux données qu'il traite. Le FHE aligne parfaitement avec les principes zero-trust et de minimisation des données prônés par le RGPD.

Ce papier est structuré comme suit. La section 2 pose les fondements théoriques du chiffrement homomorphique, depuis les rappels cryptographiques de base jusqu'à la taxonomie PHE/SHE/FHE et l'évolution historique du domaine. La section 3 analyse en profondeur les schémas FHE modernes (BFV, BGV, CKKS, TFHE) et le mécanisme de bootstrapping. La section 4 dresse un panorama de l'écosystème industriel et open-source, avec un focus particulier sur Zama et ses outils. La section 5 présente les applications concrètes dans plusieurs domaines clés. La section 6 examine avec esprit critique les limites et défis actuels du FHE. La section 7 explore les perspectives d'évolution à horizon 5-10 ans. Enfin, la section 8 conclut en répondant à notre problématique initiale.

2. Fondements théoriques du chiffrement homomorphique

2.1 Rappels cryptographiques

Avant d'aborder le chiffrement homomorphique en lui-même, il convient de rappeler les deux paradigmes fondamentaux de la cryptographie moderne, qui constituent le socle sur lequel s'appuie cette discipline.

Le chiffrement symétrique utilise une clé unique partagée entre l'émetteur et le destinataire pour chiffrer et déchiffrer les messages. Les algorithmes tels qu'AES (Advanced Encryption Standard) offrent d'excellentes performances computationnelles — typiquement de l'ordre de quelques cycles par octet sur du matériel moderne — mais nécessitent un canal sécurisé préalable pour l'échange de clé. Le chiffrement asymétrique (ou à clé publique), introduit par Diffie et Hellman en 1976 et formalisé par Rivest, Shamir et Adleman (RSA) en 1977[14], utilise une paire de clés mathématiquement liées : une clé publique pour chiffrer et une clé privée pour déchiffrer. La sécurité repose sur la difficulté computationnelle de problèmes mathématiques comme la factorisation de grands entiers (RSA) ou le logarithme discret (ElGamal, ECDSA).

La notion d'homomorphisme est empruntée à l'algèbre abstraite. Un homomorphisme est une application f entre deux structures algébriques (groupes, anneaux, corps) qui préserve les opérations : $f(a \circ b) = f(a) \circ f(b)$. Appliquée à la cryptographie, cette propriété prend un sens remarquable : si une fonction de chiffrement E est un homomorphisme, alors opérer sur les chiffrés revient à opérer sur les messages correspondants. Un cryptosystème est dit homomorphique pour l'addition si $E(m_1) + E(m_2) = E(m_1 + m_2)$, et pour la multiplication si $E(m_1) \times E(m_2) = E(m_1 \times m_2)$. Un schéma supportant à la fois l'addition et la multiplication homomorphiques — et donc tous les circuits arithmétiques — est dit pleinement homomorphique (FHE).

Pour illustrer concrètement cette propriété, considérons l'exemple de la cryptographie de Paillier[15], un schéma partiellement homomorphique pour l'addition. Si un hôpital A chiffre la valeur de glycémie d'un patient (par exemple 105 mg/dL) en E(105), et un hôpital B chiffre une deuxième mesure E(98), un serveur tiers peut calculer $E(105) \cdot E(98) = E(203)$ sans

jamais connaître les valeurs 105, 98 ou 203. L'hôpital peut ensuite déchiffrer le résultat pour obtenir 203. La somme a été calculée confidentiellement.

2.2 Taxonomie du chiffrement homomorphique

La littérature distingue classiquement trois niveaux d'homomorphisme, selon le type et le nombre d'opérations supportées.

2.2.1 Chiffrement Partiellement Homomorphique (PHE)

Un schéma PHE (Partially Homomorphic Encryption) supporte une seule opération algébrique, soit l'addition soit la multiplication, en nombre illimité. Ces schémas sont les plus anciens et les plus matures. Parmi les exemples canoniques, citons[16] :

RSA non paddé : multiplicativement homomorphique. Si $C_1 = m_1^e \bmod n$ et $C_2 = m_2^e \bmod n$, alors $C_1 \cdot C_2 = (m_1 m_2)^e \bmod n = E(m_1 \cdot m_2)$. Cela signifie que multiplier deux chiffrés RSA donne le chiffré du produit des clairs. Note : cette propriété est exploitée dans certaines attaques (attaque de multiplication RSA), d'où l'utilisation systématique de schémas avec padding (OAEP) en pratique.

ElGamal : multiplicativement homomorphique. Basé sur le problème du logarithme discret, ce schéma satisfait $E(m_1) \cdot E(m_2) = E(m_1 \cdot m_2)$ dans le groupe cyclique utilisé.

Paillier (1999)[15] : additivement homomorphique. Ce schéma, basé sur la dureté du problème de résidus composites, est très utilisé en pratique pour les applications nécessitant des sommes confidentielles (vote électronique, statistiques agrégées, scoring de crédit). Sa propriété est : $E(m_1) \cdot E(m_2) \bmod n^2 = E(m_1 + m_2 \bmod n)$.

2.2.2 Chiffrement Quelque peu Homomorphique (SHE)

Un schéma SHE (Somewhat Homomorphic Encryption) supporte à la fois l'addition et la multiplication sur les chiffrés, mais uniquement pour un nombre limité d'opérations. Cette limitation provient de l'accumulation de bruit (noise) introduit à chaque opération cryptographique. En effet, tous les schémas FHE modernes reposent sur l'introduction délibérée d'une petite erreur aléatoire dans le chiffré (pour des raisons de sécurité fondées sur la difficulté du problème LWE, détaillé à la section 3.1). Chaque opération arithmétique

augmente ce bruit, et lorsque le bruit dépasse un seuil critique, le déchiffrement devient impossible.

Les premiers schémas SHE viables furent proposés dans les travaux fondateurs de Boneh et Shacham (2004) pour les groupes bilinéaires, et de Sander, Young et Yung pour les fonctions booléennes. La véritable percée vint des travaux de Craig Gentry (2009)[1], qui proposa le premier schéma SHE basé sur les réseaux idéaux (ideal lattices), ouvrant la voie au premier vrai schéma FHE.

2.2.3 Chiffrement Pleinement Homomorphique (FHE)

Un schéma FHE (Fully Homomorphic Encryption) supporte un nombre illimité d'additions et de multiplications arbitraires sur les chiffrés, équivalent à l'évaluation de tout circuit booléen ou arithmétique. C'est ce que les cryptographes appellent depuis les années 1970 le « Saint Graal » de la cryptographie : la possibilité de déléguer des calculs arbitraires sur des données confidentielles.

L'idée clé qui permit le passage du SHE au FHE est le bootstrapping, introduit par Gentry en 2009[1] : en évaluant homomorphiquement la procédure de déchiffrement du schéma lui-même, on peut « rafraîchir » un chiffré bruité et réduire son niveau de bruit, permettant ainsi des calculs sans limite. Cette technique, bien que révolutionnaire sur le plan conceptuel, a longtemps souffert d'un coût computationnel prohibitif qui en rendait la mise en pratique difficile.

2.3 Évolution historique

L'histoire du chiffrement homomorphique s'étend sur près de cinq décennies, marquée par des avancées théoriques majeures et des défis pratiques persistants.

Les prémices (1978) : la vision initiale est formulée par Rivest, Adleman et Dertouzos dans un article technique du MIT intitulé « On Data Banks and Privacy Homomorphisms »[17], dans lequel ils imaginent la possibilité de systèmes capables d'effectuer des calculs sur des données chiffrées. Ils baptisent ce concept les « privacy homomorphisms » et en dressent les grandes lignes, sans toutefois proposer de construction concrète sécurisée. Pendant les trente années

suivantes, de nombreux chercheurs tenteront en vain de construire un tel système ou prouveront l'insécurité de ceux proposés.

La percée de Gentry (2009) : Craig Gentry, alors doctorant à Stanford, publie sa thèse révolutionnaire à STOC 2009[1] dans laquelle il propose le premier schéma FHE sécurisé. Sa construction repose sur les réseaux idéaux (ideal lattices) et introduit deux idées fondamentales : (a) un schéma SHE « bootstrappable » dont le circuit de déchiffrement est suffisamment simple pour être évalué homomorphiquement, et (b) le bootstrapping lui-même, qui recycle le schéma pour en faire un FHE. Cette publication constitue l'un des résultats les plus importants de la cryptographie du XXIe siècle.

Les générations successives de schémas FHE se sont succédé à un rythme soutenu depuis 2009, chacune apportant des améliorations majeures en termes d'efficacité et de praticabilité. La première génération (2009-2011) correspond aux travaux directs de Gentry et ses collaborateurs, basés sur les réseaux idéaux, avec des performances très limitées (minutes à heures pour des opérations simples). La deuxième génération (2011-2014) introduit les schémas BGV (Brakerski-Gentry-Vaikuntanathan, 2012)[6] et BFV (Brakerski-Fan-Vercauteren, 2012)[7], basés sur le problème LWE et Ring-LWE. Ces schémas abandonnent les réseaux idéaux pour des structures plus efficaces et introduisent la technique du « modulus switching » pour gérer le bruit. La troisième génération (2013-2016) introduit le schéma GSW (Gentry-Sahai-Waters, 2013), basé sur des matrices d'apprentissage, qui offre un bootstrapping plus simple. La quatrième génération (2016-présent) voit l'émergence de CKKS (Cheon, Kim, Kim, Song, 2017)[8] pour le calcul approché sur des nombres réels et complexes, et de TFHE (Chillotti et al., 2016-2020)[9] qui révolutionne le bootstrapping en le ramenant à une opération par porte logique en quelques millisecondes. Cette génération rend pour la première fois le FHE utilisable en pratique pour des applications réelles.

Génération	Schémas principaux	Année	Innovation clé
1ère génération	Gentry (idéaux)	2009	Premier FHE, bootstrapping conceptuel
2ème génération	BGV, BFV	2011-2012	LWE/Ring-LWE, modulus switching
3ème génération	GSW	2013	Approx. eigenvector, bootstrapping simplifié
4ème génération	CKKS, TFHE, FHEW	2016-2020	Calcul approché, PBS rapide

Tableau 1 — Évolution des générations de schémas FHE (1ère à 4ème génération).

3. Schémas FHE modernes — Analyse technique approfondie

3.1 Le problème LWE (Learning With Errors)

La sécurité de la grande majorité des schémas FHE modernes repose sur la difficulté computationnelle d'un problème introduit par Regev en 2005 : le problème LWE (Learning With Errors)[18].

Définition formelle : Soient n et q des entiers positifs (respectivement la dimension de sécurité et le module), et χ une distribution de probabilité sur Z_q appelée distribution d'erreur (généralement gaussienne discrète avec un petit écart-type). Le problème LWE est défini comme suit : étant donné un vecteur secret $s \in Z_q^n$ tiré uniformément, distinguer les paires (a_i, b_i) où $b_i = \langle a_i, s \rangle + e_i \bmod q$ (avec a_i uniformément aléatoire dans Z_q^n et e_i tiré de χ) des paires uniformément aléatoires dans $Z_q^n \times Z_q$.

Plus intuitivement, LWE est similaire à la résolution d'un système d'équations linéaires sur Z_q avec une petite erreur ajoutée à chaque équation. Sans l'erreur, le système serait trivial à résoudre par élimination gaussienne. Avec l'erreur, le problème devient computationnellement difficile, et aucun algorithme polynomial n'est connu pour le résoudre.

Ring-LWE (Ring Learning With Errors)[19], introduit par Lyubashevsky, Peikert et Regev en 2010, est une variante algébriquement plus riche qui travaille dans des anneaux polynomiaux de la forme $Z_q[X]/(X^n + 1)$ (polynômes de degré n-1 modulo $X^n + 1$ et modulo q). Cette structure permet de traiter des vecteurs entiers de taille n comme des éléments d'un anneau, ce qui réduit la taille des paramètres d'un facteur n et accélère significativement les opérations (via la transformée de nombre théorique, NTT). Ring-LWE est aujourd'hui la base de presque tous les schémas FHE pratiques.

Résistance au calcul quantique : une propriété cruciale des problèmes LWE et Ring-LWE est leur résistance supposée aux attaques par ordinateurs quantiques. L'algorithme de Shor, qui brise la sécurité de RSA et des cryptosystèmes basés sur le logarithme discret en temps polynomial quantique, ne s'applique pas aux problèmes de réseaux euclidiens. Les meilleures attaques quantiques connues contre LWE (algorithmes BKZ avec routines quantiques) n'apportent qu'un gain sous-exponentiel, laissant ces problèmes computationnellement difficiles même pour un adversaire quantique[20]. C'est pourquoi le NIST a retenu des schémas

basés sur les réseaux (CRYSTALS-Kyber, CRYSTALS-Dilithium) dans sa standardisation post-quantique[21].

[SCHÉMA : Illustration du problème LWE : à gauche, un système linéaire exact Ax = b (résolu facilement). À droite, le système bruité b = Ax + e mod q (résistant au calcul quantique). Les erreurs e sont représentées comme un bruit gaussien de faible amplitude.]

Figure 1 — Le problème LWE : comparaison entre système linéaire exact et système bruité.

3.2 Schémas BFV et BGV

Les schémas BFV (Brakerski-Fan-Vercauteren)[7] et BGV (Brakerski-Gentry-Vaikuntanathan) [6] constituent la deuxième génération de schémas FHE et restent parmi les plus utilisés en pratique pour le calcul exact sur des entiers.

Construction du schéma BFV : le schéma opère dans l'anneau de polynômes R_q = Z_q[X]/(X^n + 1). Les paramètres principaux sont : la dimension polynomiale n (typiquement 2^12 à 2^16), le module q (grand entier, souvent un produit de nombres premiers), le module de message t < q, et l'écart-type σ de la distribution d'erreur. Un chiffré est une paire $(c_0, c_1) \in R_q^2$ telle que $c_0 + c_1 \cdot s \approx \Delta m + e \pmod q$, où $\Delta = \lfloor q/t \rfloor$ est un facteur de mise à l'échelle et s est la clé secrète.

Gestion du bruit (noise budget) : chaque opération homomorphique augmente le niveau de bruit du chiffré. Le noise budget d'un chiffré fraîchement généré est typiquement de l'ordre de $\log_2(q) - \log_2(t)$ bits. Une addition homomorphique augmente modestement le bruit, tandis qu'une multiplication le fait croître quadratiquement. Lorsque le budget de bruit est épuisé, le déchiffrement est erroné. La profondeur multiplicative d'un circuit — le nombre maximum de multiplications en série — est donc le facteur limitant clé.

Différences BFV/BGV : BGV utilise une technique de « modulus switching » qui réduit le module q après chaque multiplication pour contrôler la croissance du bruit, au lieu de mettre à l'échelle les messages. BFV utilise un facteur Δ pour séparer le message du bruit. En pratique, BGV est légèrement plus efficace pour des circuits profonds, tandis que BFV est souvent préféré pour sa simplicité d'implémentation. Les deux schémas supportent le SIMD (Single Instruction Multiple Data) via le batching : on peut packager jusqu'à n/2 messages dans un seul chiffré et opérer sur tous simultanément via le Chinese Remainder Theorem (CRT), ce qui multiplie le throughput.

Cas d'usage typiques : BFV et BGV sont idéaux pour des applications nécessitant une arithmétique exacte sur des entiers modulo t, comme la détection de fraude bancaire, les requêtes sur bases de données chiffrées, ou le calcul de statistiques agrégées. Microsoft SEAL est la bibliothèque de référence pour ces schémas[10].

3.3 Schéma CKKS

Le schéma CKKS (Cheon-Kim-Kim-Song), proposé à Asiacrypt 2017[8], représente une rupture conceptuelle par rapport aux schémas BFV/BGV. Plutôt que de chercher un calcul exact sur des entiers, CKKS adopte un modèle de calcul approché sur des nombres réels et complexes — une concession qui s'avère extraordinairement bénéfique pour les applications de machine learning.

Encodage des réels : CKKS encode des vecteurs de nombres complexes (m_1, m_2, ..., m_{n/2}) ∈ C^{n/2} comme le chiffrement d'un polynôme de degré n-1. Le processus d'encodage utilise la transformée de Fourier canonique pour convertir le vecteur en un polynôme. Le déchiffrement, au lieu de chercher un résultat exact, accepte une erreur de précision bornée ε.

Calcul approché : la propriété fondamentale de CKKS est que le bruit cryptographique est traité comme une erreur d'arrondi numérique, similaire aux erreurs de virgule flottante. Après une opération homomorphique, le résultat est rescalé (divisé par un facteur) pour ramener la précision à un niveau acceptable. Cette opération de rescaling — absente des schémas BFV/BGV — est la clé de l'efficacité de CKKS pour le calcul sur les réels.

Le modèle de précision de CKKS est analogue à celui de l'arithmétique en virgule flottante IEEE 754 : on ne garantit pas un résultat exact, mais une précision relative bornée. Pour des applications de machine learning, où une précision de 16-32 bits est généralement suffisante, ce compromis est tout à fait acceptable. Des travaux récents ont cependant montré que, dans certains contextes cryptographiques, la nature approximative de CKKS peut introduire des vulnérabilités si elle n'est pas correctement gérée[22].

Applications en machine learning : CKKS est de loin le schéma le plus adapté à l'inférence et à l'entraînement de modèles de machine learning sur données chiffrées. Des opérations comme la multiplication de matrices, les fonctions d'activation (approximées par des

polynômes de bas degré), et les convolutions peuvent être évaluées efficacement. La bibliothèque Concrete ML de Zama[2], Microsoft SEAL[10] et HElib[23] proposent des interfaces haut niveau pour le ML avec CKKS.

3.4 Schéma TFHE

TFHE (Fast Fully Homomorphic Encryption over the Torus)[9], développé par Chillotti, Gama, Georgieva et Izabachène entre 2016 et 2020, adopte une approche radicalement différente des schémas précédents : plutôt que d'évaluer des circuits arithmétiques, TFHE évalue des circuits booléens en opérant porte par porte. Sa force réside dans un bootstrapping ultra-rapide — de l'ordre de quelques millisecondes — qui permet de rafraîchir le bruit après chaque opération logique.

Fondements mathématiques : TFHE opère sur le tore continu T = R/Z, représenté en pratique par des entiers modulaires à 32 ou 64 bits. Un chiffré LWE dans TFHE encode un bit (0 ou 1) comme un scalaire dans T avec un bruit gaussien ajouté. L'espace des clés est Z_q^n (typiquement n = 630 ou 1024 pour la dimension LWE interne, et N = 1024 à 4096 pour la dimension RLWE externe).

Bootstrapping programmable (PBS, Programmable Bootstrapping) : la contribution principale de TFHE est son bootstrapping programmable. En TFHE, le bootstrapping n'est pas seulement une opération de réduction du bruit : il peut simultanément évaluer n'importe quelle fonction de look-up table (LUT) sur le message. Formellement, si on dispose d'un chiffré c de m, le PBS produit un chiffré de f(m) pour une fonction f arbitraire définie par une LUT, avec un bruit réduit — le tout en une seule opération. Cela remplace n'importe quelle porte logique (AND, OR, XOR, NAND...) ainsi que des fonctions d'activation non linéaires par une seule étape de bootstrapping[9].

Performances : le bootstrapping TFHE standard prend environ 13 à 26 millisecondes sur un CPU moderne pour une porte logique, selon les paramètres de sécurité. La bibliothèque TFHE-rs de Zama[3], implémentée en Rust avec des optimisations AVX-512, a ramené ce temps à environ 6-8 ms pour 128 bits de sécurité sur du matériel x86 standard. La version GPU (cuFHE et TFHE-cuda-backend) atteint des temps de bootstrapping inférieurs à la milliseconde en parallélisant des milliers d'opérations simultanément.

TFHE est au cœur des bibliothèques phares de Zama (Concrete, TFHE-rs) et est particulièrement adapté aux applications nécessitant une évaluation de fonctions arbitraires sur des données chiffrées, comme l'exécution de smart contracts confidentiels (fhEVM) ou le traitement de signaux.

3.5 Le bootstrapping : concept, coût et optimisations

Le bootstrapping est la technique fondamentale qui permet de passer d'un schéma SHE à un schéma FHE. Son principe repose sur l'idée geniale de Gentry[1] : si le schéma est « bootstrappable » (c'est-à-dire si le circuit de déchiffrement peut être évalué homomorphiquement par le schéma lui-même), alors on peut rafraîchir un chiffré bruité en évaluant homomorphiquement sa procédure de déchiffrement pour obtenir un nouveau chiffré du même message avec un bruit réduit.

Plus formellement : soit c un chiffré de m avec un niveau de bruit proche du seuil de déchiffrement. On dispose d'un chiffré de la clé secrète sk : Enc(sk). En évaluant homomorphiquement la fonction Dec(sk, c) à l'aide de Enc(sk), on obtient Enc(Dec(sk, c)) = Enc(m) avec un niveau de bruit correspondant à un chiffré fraîchement généré. Le « paradoxe » du bootstrapping est qu'il nécessite d'utiliser le même schéma pour se déchiffrer lui-même — ce qui n'est possible que si le circuit de déchiffrement est moins profond que la capacité maximale du schéma.

Coût computationnel : le bootstrapping est l'opération la plus coûteuse du FHE. Pour les schémas BFV/BGV/CKKS, le bootstrapping prend typiquement de quelques secondes à plusieurs dizaines de secondes sur CPU, selon la profondeur souhaitée. Des techniques récentes comme le « thin bootstrapping » de CKKS (Bossuat et al., 2021) et les approches « PEGASUS » (Lu et al., 2021) l'ont significativement accéléré. Pour TFHE, le PBS constitue le bootstrapping et prend quelques millisecondes.

Optimisations récentes (2022-2026) : plusieurs directions de recherche ont permis de réduire le coût du bootstrapping. L'accélération matérielle (GPU, FPGA, ASIC — voir section 7.1) apporte des gains d'un à plusieurs ordres de grandeur. Les algorithmes améliorés comme FHEW (Ducas & Micciancio, 2015), les variants multi-valeur du PBS (Carpov et al., 2021),

et les techniques d'amortissement (amortized bootstrapping) permettent de bootstrapper plusieurs chiffrés simultanément pour un coût unitaire réduit.

4. L'écosystème industriel et open-source

4.1 Zama — Pionnier du FHE accessible

Zama est une startup française de cryptographie fondée en 2020 par Rand Hindi (PDG) et Pascal Paillier (Chief Cryptographer — co-inventeur du cryptosystème de Paillier). Basée à Paris, l'entreprise s'est donnée pour mission de rendre le FHE accessible aux développeurs sans expertise cryptographique approfondie. En 2023, Zama a levé 73 millions de dollars lors d'une série A — l'une des plus importantes levées de fonds dans le domaine de la cryptographie appliquée — avec des investisseurs tels que Protocol Labs et Multicoin Capital[24]. Ses principaux produits sont tous open-source sous licence BSD + patent grant.

4.1.1 TFHE-rs

TFHE-rs[3] est une implémentation pure Rust du schéma TFHE, optimisée pour les performances et la sécurité. Elle constitue la couche de bas niveau sur laquelle reposent tous les autres produits Zama. La version 1.0.0, publiée en février 2025, stabilise l'API haut niveau pour le backend CPU x86. TFHE-rs supporte les opérations sur des entiers chiffrés de 4 à 256 bits, les opérations booléennes, et expose une API C et WebAssembly pour une intégration dans d'autres langages et environnements.

4.1.2 Concrete

Concrete[3] est un compilateur FHE open-source qui prend en entrée des programmes Python ordinaires et les transforme automatiquement en leur équivalent FHE. Basé sur TFHE-rs et utilisant l'infrastructure de compilation LLVM, Concrete abstrait complètement la complexité cryptographique du FHE : sélection automatique des paramètres, gestion du bruit, optimisation du circuit. Un développeur peut écrire une fonction Python standard et la compiler vers FHE en quelques lignes[2]. La version 2.10 (avril 2025) introduit le support Rust et la précision multiple.

Concrete ML[2] est une extension de Concrete spécifiquement dédiée au machine learning. Elle fournit des interfaces compatibles scikit-learn pour les algorithmes classiques (régression logistique, arbres de décision, forêts aléatoires, SVM) et un support pour les réseaux de neurones (via PyTorch). L'utilisateur entraîne son modèle normalement, puis le compile vers FHE pour l'inférence confidentielle. Les benchmarks de juillet 2024 montrent que Concrete

ML atteint des performances compétitives pour l'inférence sur des modèles de classification standard, avec une précision identique au modèle en clair.

4.1.3 fhEVM

fhEVM (Fully Homomorphic Ethereum Virtual Machine)[25] est l'une des réalisations les plus ambitieuses de Zama : une machine virtuelle Ethereum étendue avec des capacités FHE. Elle permet aux développeurs de smart contracts Solidity d'utiliser des types de données chiffrés (euint8, euint32, ebool) et d'effectuer des opérations homomorphiques directement dans les smart contracts, sans que les nœuds du réseau puissent lire les valeurs manipulées. Cela ouvre la voie aux smart contracts confidentiels, aux votes électroniques on-chain et à la DeFi préservant la vie privée.

4.2 Autres acteurs majeurs

Microsoft SEAL (Simple Encrypted Arithmetic Library)[10] est la bibliothèque FHE open-source la plus utilisée dans l'industrie. Développée par l'équipe Cryptography and Privacy Research de Microsoft, SEAL implémente les schémas BFV et CKKS en C++, avec des bindings .NET et Python. Elle est particulièrement appréciée pour sa qualité de documentation, ses tutoriels, et son intégration dans l'écosystème Azure. SEAL ne supporte pas TFHE ni le bootstrapping général.

IBM HElib[23] est l'une des premières implémentations open-source de FHE, développée par Shai Halevi et Victor Shoup. Elle implémente les schémas BGV et CKKS avec un focus sur les hautes performances pour des applications BGV (notamment le SIMD via le batching NTT). HElib est utilisée dans de nombreux travaux de recherche académique et constitue une référence de facto pour les benchmarks BGV.

Google FHE Transpiler est un compilateur open-source qui traduit des programmes C++ annotés en circuits FHE évaluables par des bibliothèques comme TFHE. Bien que moins mature que Concrete, il illustre l'engagement de Google dans la démocratisation du FHE pour les développeurs C++.

Intel HE-Toolkit fournit un ensemble d'exemples et d'optimisations hardware pour SEAL et HElib, exploitant les extensions AVX-512 et HEXL (Intel Homomorphic Encryption

Acceleration Library) pour accélérer les opérations de base du FHE (NTT, multiplication polynomiale).

OpenFHE[26] est un framework open-source multi-schémas qui unifie les implémentations de BFV, BGV, CKKS, FHEW et TFHE au sein d'une API commune. Successeur de PALISADE, il est développé par une coalition d'universités et d'entreprises sous la direction de Duality Technologies. OpenFHE supporte également le bootstrapping pour CKKS et FHEW.

Bibliothèque	Éditeur	Schémas	Langage	Bootstrapping
SEAL	Microsoft	BFV, CKKS	C++, Python	Non (CKKS limité)
HElib	IBM	BGV, CKKS	C++	Partiel (CKKS)
Concrete / TFHE-rs	Zama	TFHE	Python, Rust, C	Oui (PBS complet)
OpenFHE	Communauté	BFV, BGV, CKKS, TFHE	C++, Python	Oui (CKKS, FHEW)
Google FHE	Google	TFHE	C++	Oui
Lattigo	Tune Insight	BFV, BGV, CKKS, RLWE	Go	Oui (CKKS)

Tableau 2 — Comparatif des principales bibliothèques FHE open-source.

4.3 Standardisation

La standardisation du FHE est un chantier encore largement ouvert. Plusieurs initiatives en cours témoignent de la prise de conscience de la communauté internationale quant à la nécessité d'un cadre normatif.

HomomorphicEncryption.org est un consortium industriel fondé en 2017 réunissant des acteurs académiques et industriels (Microsoft, IBM, Intel, MIT, Stanford, etc.) avec pour mission de standardiser les paramètres de sécurité et les interfaces du FHE. Le consortium a publié un standard HE[27] définissant les niveaux de sécurité recommandés (128, 192 et 256 bits) et les paramètres LWE correspondants (dimension n, module q, distribution d'erreur). Ce document de référence est largement adopté par les bibliothèques FHE pour le choix de paramètres sécurisés.

Le NIST (National Institute of Standards and Technology) a lancé en 2023 un appel à contributions pour la standardisation du FHE[28], dans le sillage de sa démarche de standardisation post-quantique. Le processus est encore en cours, mais plusieurs candidats sont en cours d'évaluation.

La NCSC (National Cyber Security Centre) du Royaume-Uni a publié en 2025 un white paper sur la cryptographie avancée[29], incluant une section substantielle sur le FHE. La NCSC adopte une position prudente mais positive : le FHE est reconnu comme une technologie prometteuse répondant à des besoins réels de confidentialité des données, mais son adoption en environnement gouvernemental est conditionnée à l'existence de standards formels validés et à des déploiements en production éprouvés. La NCSC souligne en particulier l'absence de garanties d'intégrité du FHE (qui ne protège que la confidentialité, pas l'authenticité) et la complexité du paramétrage sécurisé.

5. Applications concrètes et cas d'usage

5.1 Santé et données médicales

Le domaine de la santé constitue l'un des cas d'usage les plus convaincants et les plus urgents du FHE. Les données médicales sont parmi les plus sensibles qui existent — elles engagent non seulement la vie privée des individus, mais peuvent affecter leur assurabilité, leur emploi et leur dignité. Simultanément, l'analyse de grandes cohortes de patients est indispensable à la recherche médicale, à l'épidémiologie et au développement de traitements personnalisés.

La recherche sur les dossiers médicaux chiffrés illustre parfaitement la valeur ajoutée du FHE. Un consortium d'hôpitaux souhaitant entraîner un modèle prédictif de détection du diabète sur les données combinées de plusieurs établissements se heurte classiquement à des obstacles réglementaires (RGPD, secret médical) et à la réticence des patients à partager leurs données. Avec le FHE, chaque hôpital chiffre ses données localement, un serveur central effectue les calculs d'apprentissage sur les données chiffrées, et les paramètres du modèle sont partagés sans que les données individuelles ne soient jamais exposées. Des études de cas publiées en 2025[30] montrent que cette approche permet d'atteindre une précision de modèle identique à celle obtenue sur données en clair (90,16 % de précision pour la prédiction de maladies cardiaques avec Paillier dans un cadre de régression logistique).

L'analyse génomique constitue un autre domaine d'application majeur. L'ADN est une donnée extrêmement sensible — il ne peut pas être modifié et révèle des informations sur toute la famille biologique. Des projets comme iDASH (integrating Data for Analysis, Anonymization and SHaring) organisent depuis 2014 des compétitions annuelles de machine learning sur données génomiques chiffrées par FHE. Les résultats montrent que des algorithmes GWAS (Genome-Wide Association Studies) peuvent être exécutés sur des cohortes de milliers de patients avec des surcoûts acceptables.

La conformité réglementaire est un moteur supplémentaire d'adoption. Le RGPD impose le principe de minimisation des données (article 5) et des garanties techniques appropriées. Le Health Data Hub français, plateforme nationale d'hébergement et d'analyse des données de santé, a identifié le FHE comme une technologie d'avenir pour permettre des analyses secondaires des données de santé sans exposition des données individuelles.

5.2 Finance et secteur bancaire

Le secteur financier génère et traite des volumes colossaux de données sensibles : transactions, scores de crédit, portefeuilles d'investissement, informations comportementales. La conformité réglementaire (RGPD, directive MiFID II, réglementation anti-blanchiment) impose des obligations croissantes de protection des données, tandis que la compétition et la lutte contre la fraude exigent des analyses de plus en plus sophistiquées.

La détection de fraude sur données chiffrées est un cas d'usage particulièrement attractif. Aujourd'hui, les banques partagent des informations sur les fraudes connues via des consortia sectoriels, mais ce partage est limité par la sensibilité des données (informations sur les clients, les patterns de fraude). Avec le FHE, une banque peut envoyer des données de transaction chiffrées à un serveur de scoring mutualisé, qui calcule un score de risque homomorphiquement et renvoie le résultat chiffré. Selon des estimations récentes, plus de 250 millions de transactions financières chiffrées ont été traitées via des systèmes FHE ou semi-homomorphiques en 2023[31].

Le scoring de crédit privacy-preserving représente une autre application majeure. Un modèle de scoring de crédit évalue la solvabilité d'un emprunteur à partir de ses données financières. Avec le FHE, un candidat peut soumettre ses données chiffrées à une banque, qui calcule le score homomorphiquement et ne reçoit que le résultat binaire (accordé/refusé), sans jamais accéder aux données brutes. Inversement, une banque peut évaluer le score d'un client sur un modèle propriétaire d'une fintech sans révéler les paramètres du modèle ni les données du client.

La lutte anti-blanchiment (AML) bénéficie également du FHE pour la surveillance collaborative. Des entités financières concurrentes peuvent collaborer pour détecter des schémas de blanchiment qui traversent plusieurs institutions sans partager les données transactionnelles en clair — une approche réglementairement favorable et techniquement réalisable avec des schémas additifs comme Paillier ou CKKS.

5.3 Machine learning confidentiel

Le machine learning confidentiel (Confidential ML ou Privacy-Preserving Machine Learning, PPML) est le domaine qui a connu la croissance la plus rapide en termes d'applications FHE.

La tension entre la nécessité d'accéder à des données riches pour entraîner des modèles performants et les exigences de confidentialité des données constitue l'un des problèmes centraux de l'IA moderne.

L'inférence FHE (FHE Inference) consiste à exécuter un modèle ML pré-entraîné sur des données chiffrées par l'utilisateur. Le serveur hébergeant le modèle effectue l'inférence homomorphiquement et renvoie le résultat chiffré. L'utilisateur déchiffre le résultat localement. La propriété remarquable est double : l'utilisateur ne révèle pas ses données au serveur, et le serveur ne révèle pas son modèle à l'utilisateur (model hiding). Concrete ML de Zama propose une implémentation complète de ce scénario pour des modèles scikit-learn et des réseaux de neurones PyTorch. Le principal défi est l'approximation des fonctions d'activation non-polynomiales (ReLU, sigmoid) par des polynômes de bas degré pour les rendre évaluables par FHE.

L'entraînement FHE (FHE Training) est techniquement beaucoup plus ambitieux et computationnellement coûteux. Il s'agit d'effectuer l'ensemble du processus de descente de gradient (forward pass + backward pass + mise à jour des paramètres) sur des données chiffrées. Des travaux récents (Han et al., 2019 ; Nandakumar et al., 2019) ont démontré la faisabilité pour des modèles simples (régression logistique, petits réseaux de neurones) mais le coût computationnel reste prohibitif pour des modèles de grande taille.

Comparaison avec d'autres approches PPML : le FHE n'est pas la seule technique de machine learning préservant la vie privée. Le calcul multipartite sécurisé (MPC), la confidentialité différentielle (Differential Privacy, DP), et les environnements d'exécution de confiance (TEE, Trusted Execution Environments) offrent des alternatives avec des compromis différents. La DP offre des garanties mathématiques formelles sur la fuite d'information individuelle mais dégrade la précision du modèle. Le MPC distribue le calcul entre plusieurs parties sans serveur central de confiance mais nécessite une communication importante. Les TEE (comme Intel SGX) offrent de bonnes performances mais reposent sur la confiance matérielle et ont été sujets à des attaques side-channel. Le FHE offre la garantie la plus forte de confidentialité computationnelle sans TEE, mais au prix de performances significativement dégradées.

Technique	Garantie de confidentialité	Performances	Communication	Modèle de confiance
FHE	Très élevée (crypto)	Faible (100x-1M×)	Faible	Aucun tiers de confiance
MPC	Élevée (info-théorique)	Moyenne (10x-100×)	Élevée	Majorité honnête
TEE (SGX)	Élevée (matérielle)	Bonne (1.5x-10×)	Faible	Fabricant hardware
Diff. Privacy	Formelle (ε,δ)	Bonne	Faible	Aucun
FHE + MPC	Maximale	Très faible	Moyenne	Aucun

Tableau 3 — Comparaison des approches de Privacy-Preserving Machine Learning (PPML).

5.4 Cloud computing et souveraineté des données

Le cloud computing non-trusté représente la menace fondamentale à laquelle le FHE répond le plus directement. Dans le modèle de cloud computing actuel, un utilisateur délègue des calculs à un prestataire cloud (AWS, Azure, GCP) et doit lui faire confiance pour la confidentialité de ses données. Cette confiance est limitée par plusieurs menaces réelles : compromission du prestataire par un attaquant (violation de données), accès par des employés malveillants, injonctions légales (Cloud Act américain permettant l'accès aux données hébergées par des entreprises américaines même en Europe).

Le FHE permet de rompre cette dépendance de confiance : les données sont chiffrées côté client avant envoi au cloud, le cloud effectue les calculs sur les données chiffrées, et le client déchiffre les résultats. Le prestataire cloud n'a accès qu'à des données chiffrées qu'il ne peut pas déchiffrer, même en cas de compromission. Ce modèle est particulièrement pertinent pour les données soumises à des exigences de souveraineté (données gouvernementales, données de défense, données médicales) qui doivent être traitées dans un cloud non-souverain pour des raisons économiques ou techniques.

Comme alternative aux enclaves sécurisées (Intel SGX, ARM TrustZone), le FHE présente l'avantage de ne reposer sur aucune hypothèse de confiance matérielle. SGX a été affecté par plusieurs vulnérabilités side-channel (Spectre, Meltdown, SGAxe) qui ont compromis ses garanties de sécurité. Le FHE, basé sur des problèmes mathématiques, ne peut pas être compromis par des attaques hardware de ce type.

5.5 Blockchain et Web3

La blockchain est un domaine où la tension entre transparence (condition d'une vérifiabilité décentralisée) et confidentialité (protection des utilisateurs) est particulièrement aigüe. Les blockchains publiques comme Ethereum exposent par design l'ensemble des données transactionnelles, permettant certes la vérification par tous les participants, mais rendant impossible la construction d'applications financières confidentielles.

L'EVM de Zama (fhEVM)[25] résout ce dilemme en étendant la machine virtuelle Ethereum avec des types de données FHE. Les smart contracts peuvent manipuler des valeurs chiffrées (euint32 pour des entiers chiffrés, ebool pour des booléens chiffrés) et effectuer des opérations homomorphiques sur celles-ci. Les nœuds du réseau exécutent les contrats sur des données chiffrées sans jamais accéder aux valeurs en clair. Les cas d'usage sont nombreux : DEX (exchanges décentralisés) où les ordres restent confidentiels jusqu'à leur exécution, systèmes de vote on-chain vérifiables et secrets, jeux blockchain où l'état privé des joueurs n'est pas révélé aux adversaires.

Les votes électroniques vérifiables et confidentiels constituent un autre domaine d'application emblématique. Un système de vote électronique idéal doit simultanément garantir la confidentialité du vote (personne ne doit savoir pour qui vous avez voté), l'intégrité (les votes ne peuvent pas être manipulés) et la vérifiabilité (tout électeur peut vérifier que son vote a été compté). Le FHE combiné aux preuves à divulgation nulle de connaissance (ZKP) permet d'approximer cet idéal : les votes sont chiffrés homomorphiquement, leur agrégation est effectuée sans déchiffrement, et le résultat final est seul déchiffré.

6. Limites, défis et critiques

6.1 Performances — L'obstacle principal

L'obstacle le plus significatif à l'adoption généralisée du FHE reste son coût computationnel. Malgré des progrès spectaculaires au cours des quinze dernières années, un écart de performance considérable demeure entre le calcul sur données en clair et le calcul FHE.

Pour quantifier cet écart, considérons des benchmarks concrets issus de la littérature récente (2024-2025)[32]. Pour une addition simple de deux entiers : environ 1 nanoseconde en clair contre 100-500 microsecondes en FHE TFHE (facteur 100 000). Pour une multiplication : 2-5 nanosecondes en clair contre 1-10 millisecondes en FHE BFV (facteur 500 000 à 5 000 000). Pour une inférence de réseau de neurones simple (10 neurones) sur CKKS : environ 1 milliseconde en clair contre 1-10 secondes en FHE (facteur 1 000 à 10 000). Pour un bootstrapping TFHE : opération sans équivalent en clair, prenant 6 à 26 millisecondes selon les paramètres.

L'expansion des données (ciphertext expansion) est un second problème pratique majeur. Un chiffré FHE est beaucoup plus volumineux que la donnée en clair correspondante. En BFV/CKKS, un chiffré occupe typiquement 40 à 200 kilooctets pour quelques octets de données utiles — un facteur d'expansion de l'ordre de 10 000. En TFHE, un chiffré LWE occupe 2-4 kilooctets pour un seul bit ou un petit entier. Cette expansion a des implications directes sur les besoins en bande passante réseau et en mémoire vive, constituant un goulot d'étranglement pour les applications nécessitant de traiter de grands volumes de données.

La consommation mémoire est également significative. Une application FHE typique peut nécessiter plusieurs gigaoctets de RAM pour stocker les clés de chiffrement (clé publique, clé de réévaluation/relinearization key, clé de rotation/Galois keys), les chiffrés et les données intermédiaires. Pour CKKS avec des paramètres standards à 128 bits, la clé de réévaluation seule peut peser plusieurs centaines de mégaoctets.

6.2 Complexité d'implémentation

La mise en œuvre correcte d'un schéma FHE requiert une expertise cryptographique de haut niveau, difficile à trouver sur le marché. Le choix des paramètres — dimension polynomiale

n, modules q et t, distribution d'erreur, profondeur multiplicative — est un exercice délicat qui doit simultanément satisfaire les exigences de sécurité (niveaux définis par HomomorphicEncryption.org), de correctitude (budget de bruit suffisant pour le calcul voulu) et de performance. Un mauvais choix peut rendre le système soit insécurisé (paramètres trop petits), soit incorrect (budget de bruit insuffisant), soit inefficace (paramètres trop grands).

Les risques de mauvaise implémentation sont réels et potentiellement graves. Une gestion incorrecte du bruit peut conduire à des erreurs de déchiffrement silencieuses. Des oracles de déchiffrement mal configurés peuvent leaker des informations sur la clé secrète (attaques de type IND-CCA). Des implémentations non-constantes en temps peuvent être vulnérables aux attaques par canaux auxiliaires (timing attacks). Des travaux récents (Li et Micciancio, 2021) [22] ont montré que le caractère approché de CKKS peut être exploité dans des scénarios adversariaux pour récupérer des informations sur les données en clair à partir des résultats des calculs homomorphiques, si des mesures de protection appropriées ne sont pas prises.

Le manque de développeurs formés est une contrainte pratique significative. Le FHE reste une discipline de niche, enseignée dans peu de formations universitaires et mal documentée dans les ressources pédagogiques grand public. Des initiatives comme Concrete de Zama (qui abstrait la complexité cryptographique derrière une interface Python familière) et les tutoriels de SEAL tentent de réduire cette barrière, mais la maturité de l'écosystème ne se compare pas encore à celle des bibliothèques de machine learning ou de cryptographie classique.

6.3 Maturité et confiance

Malgré les avancées spectaculaires depuis 2009, le FHE reste une technologie en phase de maturation. Les déploiements en production à grande échelle sont encore rares, et les retours d'expérience industriels limités. La plupart des cas d'usage publiés sont des prototypes de recherche ou des preuves de concept, non des systèmes opérationnels soumis à des volumes de production réels.

L'absence de standards formels validés est un frein à l'adoption institutionnelle. Le processus de standardisation NIST pour le FHE est encore en cours, et les standards de HomomorphicEncryption.org, bien qu'utiles, n'ont pas le même poids normatif qu'un FIPS (Federal Information Processing Standard) ou un standard ISO. Les organisations soumises à

des exigences réglementaires strictes (banques, hôpitaux, administrations) hésitent à adopter des technologies non standardisées.

La position prudente de la NCSC[29] reflète l'état d'esprit des agences de sécurité nationales : le FHE est reconnu comme une avancée théorique majeure répondant à des besoins réels, mais son adoption en environnement gouvernemental sensible nécessite une maturité plus grande. La NCSC souligne en particulier que le FHE ne fournit pas de garanties d'intégrité des données — un adversaire actif pourrait manipuler les chiffrés pour produire des résultats de calcul erronés — et que cette limitation doit être compensée par des mécanismes complémentaires (signatures, preuves ZKP).

La communauté cryptographique maintient également une saine vigilance sur la sécurité à long terme des schémas basés sur LWE. Bien que le problème LWE soit largement supposé difficile, les meilleures attaques algorithmiques connues (BKZ avec routines LLL améliorées) évoluent. Des travaux récents (2022-2025) ont continué à améliorer marginalement l'efficacité de ces attaques, nécessitant des ajustements réguliers des niveaux de paramètres recommandés par HomomorphicEncryption.org.

6.4 Menace quantique et sécurité post-quantique

Une question fréquemment posée est la suivante : si le FHE est basé sur des problèmes post-quantiques (LWE, Ring-LWE), est-il intrinsèquement résistant au calcul quantique ? La réponse est nuancée.

Les meilleures attaques quantiques connues contre LWE utilisent des variantes quantiques des algorithmes de base de réseaux (BKZ quantique). Ces algorithmes fournissent un avantage sub-exponentiel — et non polynomial comme l'algorithme de Shor — de sorte que LWE reste difficile pour un ordinateur quantique avec des paramètres suffisamment grands. Le NIST a retenu Kyber (basé sur Module-LWE) et Dilithium (basé sur Module-LWE) dans sa standardisation post-quantique, ce qui constitue une validation implicite de la résistance de LWE au calcul quantique avec des paramètres appropriés[21].

Cependant, une distinction importante doit être faite entre la résistance quantique du problème LWE sous-jacent et la sécurité globale d'un système FHE. Des composantes additionnelles comme les clés de rotation ou certaines structures algébriques pourraient théoriquement être

exploitées par des algorithmes quantiques encore non découverts. La communauté cryptographique maintient une surveillance active des avancées en cryptanalyse quantique des schémas de réseaux, et les paramètres recommandés par HomomorphicEncryption.org intègrent des marges de sécurité explicitement conçues pour absorber les progrès futurs en cryptanalyse.

7. Perspectives et futur du FHE

7.1 Accélération matérielle

L'accélération matérielle représente la voie la plus directe vers des performances FHE utilisables pour des applications temps-réel. Le programme DARPA DPRIVE (Data Protection in Virtual Environments), lancé en 2020 avec un budget de plusieurs dizaines de millions de dollars, a financé le développement de puces ASIC dédiées au FHE[33]. Trois équipes ont participé : Duality Technologies, Galois Inc. et Intel.

Le projet HERACLES d'Intel, développé dans le cadre de DPRIVE, vise à concevoir une architecture near-memory spécialisée pour les opérations fondamentales du FHE : transformée de nombre théorique (NTT), multiplication polynomiale, et opérations d'addition/ soustraction de polynômes. Les résultats préliminaires montrent un gain de plus de trois ordres de magnitude (1 000×) par rapport à une implémentation logicielle optimisée sur CPU x86. Si les objectifs finaux du programme sont atteints d'ici 2025, les performances FHE devraient approcher celles du calcul en clair pour de nombreuses applications pratiques[33].

Parallèlement, des travaux de janvier 2025 publiés par des chercheurs de Cornell, Google, MIT et Georgia Tech ont démontré comment les puces d'accélération IA existantes — en particulier les TPU (Tensor Processing Units) de Google — peuvent être réutilisées pour accélérer le FHE. En recadrant les opérations FHE comme des opérations matricielles, les auteurs montrent des gains de performance significatifs sans nécessiter de hardware dédié, ce qui pourrait accélérer l'adoption industrielle à court terme.

Les FPGA (Field-Programmable Gate Arrays) constituent une voie intermédiaire entre les CPU généralistes et les ASIC spécialisés. Des implémentations FPGA des schémas BFV, CKKS et TFHE ont démontré des gains de 10 à 100× par rapport au CPU, avec une flexibilité de reconfiguration appréciable pour des environnements de recherche. Des projets comme CryptoNets-FPGA et les travaux de F72 (startup spécialisée en FHE hardware) explorent cette direction.

7.2 Convergence avec d'autres technologies

Le FHE ne s'inscrit pas dans un vide technologique : sa convergence avec d'autres paradigmes cryptographiques et de calcul distribué crée des possibilités inédites.

FHE + MPC (Multi-Party Computation) : le calcul multipartite sécurisé permet à plusieurs parties de calculer conjointement une fonction de leurs données privées sans que chacune révèle ses données aux autres. Combiné avec le FHE, ce paradigme permet un calcul collaboratif ultra-sécurisé où non seulement les données sont chiffrées mais le déchiffrement lui-même nécessite la coopération de plusieurs parties (secret sharing + FHE). Des bibliothèques comme SCALE-MAMBA et des travaux académiques récents explorent ces hybridations pour des applications de gestion de clés distribuées et de calcul fédéré à haute assurance.

FHE + Preuves à Divulgation Nulle de Connaissance (ZKP) : les preuves ZKP permettent à une partie de prouver qu'un calcul a été effectué correctement sans révéler les données d'entrée. Combinées au FHE, elles permettent de garantir non seulement la confidentialité (fournie par le FHE) mais aussi la vérifiabilité du calcul (fournie par les ZKP). Un serveur peut ainsi prouver qu'il a correctement exécuté un modèle ML sur des données chiffrées, sans que ni les données ni les paramètres du modèle ne soient révélés. Des travaux récents (Boneh et al., 2023) formalisent ce paradigme sous le terme de « Verifiable FHE ».

FHE + TEE : une approche hybride combine les garanties du FHE (confidentialité cryptographique) avec celles des TEE (performance et flexibilité). Les données sensibles sont traitées dans un TEE après avoir été transmises chiffrées (via FHE), offrant une double couche de protection. Cette approche pragmatique peut être appropriée dans des contextes où la performance est critique et le niveau de menace ne justifie pas les surcoûts du FHE pur.

7.3 Démocratisation du FHE

La démocratisation du FHE — le rendre accessible à des développeurs non-cryptographes — est un enjeu aussi important que les avancées théoriques. Plusieurs leviers sont actionnés simultanément.

Les compilateurs FHE de haut niveau constituent l'approche la plus prometteuse. Concrete de Zama[2] et le Google FHE Transpiler permettent à un développeur d'écrire un programme Python ou C++ standard et de le compiler vers FHE automatiquement, sans choisir

manuellement les paramètres cryptographiques. Ces outils prennent en charge la sélection de paramètres, la gestion du bruit, et l'optimisation du circuit. Bien qu'encore limités en termes d'expressivité (tous les programmes ne sont pas compilables vers FHE), ils ont radicalement réduit la barrière à l'entrée.

La formation et l'éducation progressent également. Des MOOCs dédiés au FHE, des tutoriels interactifs dans les documentations de Zama et SEAL, et des cours universitaires spécialisés (ETH Zurich, MIT, Stanford) développent le vivier de compétences dans ce domaine. La démocratisation passe aussi par une meilleure intégration dans les frameworks ML existants (PyTorch, scikit-learn) via des bibliothèques comme Concrete ML.

Les prévisions d'adoption à horizon 5-10 ans sont prudemment optimistes. La combinaison de l'accélération matérielle (ASIC FHE dédiés, réutilisation des TPU), de l'amélioration algorithmique (bootstrapping plus rapide, meilleures approximations polynomiales) et de la maturation de l'écosystème (standards NIST, déploiements pilotes en production) devrait permettre de réduire l'overhead FHE d'un à deux ordres de magnitude supplémentaires d'ici 2030. Pour des applications comme l'inférence ML sur données de santé ou le scoring de crédit, des délais de quelques secondes plutôt que quelques minutes seraient déjà suffisants pour de nombreux cas d'usage industriels.

8. Conclusion

Ce papier a présenté un état de l'art complet du chiffrement homomorphique, depuis ses fondements théoriques jusqu'aux perspectives industrielles, en réponse à la problématique centrale suivante : comment effectuer des calculs sur des données chiffrées sans jamais les déchiffrer ?

La réponse qu'apporte le FHE à cette question est à la fois élégante et révolutionnaire. En exploitant les propriétés algébriques des problèmes de réseaux euclidiens — en particulier LWE et Ring-LWE — les schémas modernes comme BFV, BGV, CKKS et TFHE permettent d'effectuer des opérations arithmétiques arbitraires sur des données chiffrées, avec des garanties de sécurité fondées sur des problèmes mathématiques supposés difficiles même pour un ordinateur quantique. Le bootstrapping, introduit par Gentry en 2009[1], est le mécanisme clé qui élève le chiffrement partiellement homomorphique au rang de chiffrement pleinement homomorphique, permettant des calculs sans limitation de profondeur.

L'écosystème autour du FHE a connu une maturité remarquable au cours des cinq dernières années. Des bibliothèques comme Concrete de Zama, Microsoft SEAL, HElib et OpenFHE offrent aujourd'hui des outils exploitables, même si leur utilisation en production reste réservée à des cas d'usage spécifiques où le surcoût computationnel est acceptable. Les applications dans les domaines de la santé, de la finance et du machine learning confidentiel démontrent la pertinence pratique du FHE pour des enjeux réels de cybersécurité et de conformité réglementaire.

Les limites actuelles — overhead computationnel de l'ordre de 10 000 à 1 000 000×, expansion des données, complexité du paramétrage — sont réelles et ne doivent pas être minimisées. La position prudente des agences comme la NCSC[29] est justifiée et reflète l'état effectif de maturité du domaine. Cependant, l'accélération matérielle en cours (DARPA DPRIVE, Intel HERACLES) et les avancées algorithmiques suggèrent que ces limites ne sont pas insurmontables.

Pour les professionnels de la cybersécurité, le message pratique est clair : le FHE n'est pas encore une solution générale prête à l'emploi, mais il est déjà opérationnel pour des cas d'usage ciblés où la confidentialité est critique et où les contraintes de performance peuvent

être assouplies. Les architectes de sécurité doivent l'intégrer dans leur boîte à outils conceptuelle et évaluer sa pertinence au cas par cas, en tenant compte des alternatives (MPC, TEE, différentiel privacy).

Le FHE représente, à terme, une révolution potentielle de l'architecture de confiance numérique. Dans un monde où la souveraineté des données, la compliance RGPD et la menace quantique sont des préoccupations centrales, la capacité à déléguer des calculs sans déléguer la confiance constitue un changement de paradigme fondamental. Les prochaines années verront très probablement le FHE passer du statut de curiosité académique à celui de composante essentielle des architectures de sécurité modernes — un pilier de la cybersécurité de demain.

Bibliographie

[1] C. Gentry, "A Fully Homomorphic Encryption Scheme," Ph.D. Dissertation, Stanford University, 2009, and "Fully Homomorphic Encryption Using Ideal Lattices," in Proc. 41st Annual ACM Symp. Theory of Computing (STOC), Bethesda, MD, 2009, pp. 169-178.

[2] Zama, "Concrete ML: Privacy-Preserving Machine Learning," Official Documentation, https://docs.zama.ai/concrete-ml, accessed March 2026.

[3] Zama, "TFHE-rs: A Pure Rust Implementation of TFHE," GitHub, https://github.com/zama-ai/tfhe-rs, 2021-2025 ; and "Concrete: TFHE Compiler," https://github.com/zama-ai/concrete, accessed March 2026.

[4] National Cyber Security Centre (NCSC), "Advanced Cryptography: New Approaches to Data Privacy," White Paper, NCSC.GOV.UK, 2025.

[5] HomomorphicEncryption.org, "Homomorphic Encryption Standard," Draft Standard, https://homomorphicencryption.org/, v1.1, 2020.

[6] Z. Brakerski, C. Gentry, and V. Vaikuntanathan, "(Leveled) Fully Homomorphic Encryption without Bootstrapping," in Proc. 3rd Innovations in Theoretical Computer Science Conference (ITCS), Cambridge, MA, 2012, pp. 309-325.

[7] J. Fan and F. Vercauteren, "Somewhat Practical Fully Homomorphic Encryption," Cryptology ePrint Archive, Report 2012/144, 2012.

[8] J. H. Cheon, A. Kim, M. Kim, and Y. Song, "Homomorphic Encryption for Arithmetic of Approximate Numbers," in Proc. Advances in Cryptology — Asiacrypt 2017, Hong Kong, 2017, pp. 409-437.

[9] I. Chillotti, N. Gama, M. Georgieva, and M. Izabachène, "TFHE: Fast Fully Homomorphic Encryption over the Torus," Journal of Cryptology, vol. 33, no. 1, pp. 34-91, Jan. 2020.

[10] Microsoft Research, "Microsoft SEAL: Simple Encrypted Arithmetic Library," GitHub, https://github.com/microsoft/SEAL, version 4.1, 2023.

[11] Gartner Inc., "Gartner Forecasts Worldwide Public Cloud End-User Spending to Reach Nearly $600 Billion in 2023," Press Release, Apr. 2023.

[12] European Parliament and Council, "Regulation (EU) 2016/679 of 27 April 2016 on the protection of natural persons with regard to the processing of personal data (GDPR)," Official Journal of the European Union, 2016.

[13] J. Kindervag, "Build Security in Maturity Model: Zero Trust," Forrester Research Report, 2010.

[14] R. L. Rivest, A. Shamir, and L. Adleman, "A Method for Obtaining Digital Signatures and Public-Key Cryptosystems," Commun. ACM, vol. 21, no. 2, pp. 120-126, 1978.

[15] P. Paillier, "Public-Key Cryptosystems Based on Composite Degree Residuosity Classes," in Proc. Advances in Cryptology — Eurocrypt 1999, Prague, 1999, pp. 223-238.

[16] T. ElGamal, "A Public Key Cryptosystem and a Signature Scheme Based on Discrete Logarithms," IEEE Trans. Inf. Theory, vol. 31, no. 4, pp. 469-472, 1985.

[17] R. L. Rivest, L. Adleman, and M. L. Dertouzos, "On Data Banks and Privacy Homomorphisms," in Foundations of Secure Computation, R. A. DeMillo et al., Eds., Academic Press, 1978, pp. 169-179.

[18] O. Regev, "On Lattices, Learning with Errors, Random Linear Codes, and Cryptography," J. ACM, vol. 56, no. 6, pp. 1-40, 2009.

[19] V. Lyubashevsky, C. Peikert, and O. Regev, "On Ideal Lattices and Learning with Errors over Rings," in Proc. Advances in Cryptology — Eurocrypt 2010, Monaco, 2010, pp. 1-23.

[20] M. R. Albrecht et al., "Estimating Quantum Speedups for Lattice Sieves," in Proc. Advances in Cryptology — Asiacrypt 2020, Daejeon, 2020, pp. 583-613.

[21] NIST, "Module-Lattice-Based Key-Encapsulation Mechanism Standard (FIPS 203)," Federal Information Processing Standards Publication, 2024.

[22] B. Li and D. Micciancio, "On the Security of Homomorphic Encryption on Approximate Numbers," in Proc. Advances in Cryptology — Eurocrypt 2021, Zagreb, 2021, pp. 648-677.

[23] S. Halevi and V. Shoup, "HElib: An Implementation of Homomorphic Encryption," GitHub, https://github.com/homenc/HElib ; and "Algorithms in HElib," in Proc. CRYPTO 2014, Santa Barbara, 2014.

[24] Zama, "Zama Raises $73M Series A to Accelerate the Deployment of Fully Homomorphic Encryption," Press Release, Zama.ai, Aug. 2023.

[25] Zama, "fhEVM: Confidential Smart Contracts on EVM Using Fully Homomorphic Encryption," Documentation, https://docs.zama.ai/fhevm, 2023-2025.

[26] A. Al Badawi et al., "OpenFHE: Open-Source Fully Homomorphic Encryption Library," in Proc. 10th ACM Workshop on Encrypted Computing and Applied Homomorphic Cryptography (WAHC), Los Angeles, 2022.

[27] M. Albrecht et al., "Homomorphic Encryption Security Standard," HomomorphicEncryption.org, Toronto, Canada, Nov. 2018 (updated 2021).

[28] NIST, "Homomorphic Encryption Standardization Workshop," National Institute of Standards and Technology, 2023, https://csrc.nist.gov/events/2023/3rd-he-standardization-workshop.

[29] National Cyber Security Centre (NCSC), "Advanced Cryptography," White Paper, https://www.ncsc.gov.uk/whitepaper/advanced-cryptography, 2025.

[30] A. Kumar et al., "Homomorphic Encryption for Secure and Scalable Predictive Healthcare Analytics," Life Cycle Reliability and Safety Engineering, Springer Nature, 2025.

[31] T. Borup et al., "Encrypted Intelligence: A Comparative Analysis of Homomorphic Encryption Frameworks for Privacy-Preserving AI," ScienceDirect (Elsevier), 2025.

[32] K. Tsuji et al., "Comparison of FHE Schemes and Libraries for Efficient Cryptographic Processing," in Proc. ICNC 2024, 2024, pp. 584-590.

[33] DARPA, "Data Protection in Virtual Environments (DPRIVE) Program," https://www.darpa.mil/program/data-protection-in-virtual-environments, 2020-2025 ; and Intel Community, "Intel Completes DARPA DPRIVE Phase One Milestone for a Fully Homomorphic Encryption Platform," 2023.

Annexes

Annexe A — Glossaire des termes techniques

Bootstrapping : Technique permettant de réduire le bruit accumulé dans un chiffré FHE en évaluant homomorphiquement la procédure de déchiffrement, transformant un schéma SHE en schéma FHE.

Chiffrement Homomorphique Partiel (PHE) : Schéma supportant uniquement une opération algébrique (addition OU multiplication) sur les données chiffrées, sans limitation de nombre d'opérations.

Chiffrement Pleinement Homomorphique (FHE) : Schéma supportant toute opération arithmétique ou logique arbitraire sur des données chiffrées, en nombre illimité.

Ciphertext Expansion : Rapport entre la taille d'un chiffré FHE et la taille de la donnée en clair correspondante (typiquement 10 000 à 100 000×).

CKKS : Schéma FHE de Cheon-Kim-Kim-Song (2017), optimisé pour le calcul approché sur des nombres réels et complexes, particulièrement utilisé pour le machine learning confidentiel.

Concrete : Compilateur FHE open-source de Zama qui transforme automatiquement des programmes Python en circuits FHE utilisant le schéma TFHE.

fhEVM : Machine Virtuelle Ethereum étendue avec des capacités FHE, permettant l'exécution de smart contracts Solidity sur des données chiffrées.

LWE (Learning With Errors) : Problème mathématique fondant la sécurité des schémas FHE modernes, consistant à distinguer des équations linéaires bruitées de données aléatoires. Supposé difficile même pour un ordinateur quantique.

Noise Budget : Budget de bruit disponible dans un chiffré FHE. Chaque opération homomorphique consomme du budget de bruit. Quand le budget est épuisé, le déchiffrement est incorrect.

PBS (Programmable Bootstrapping) : Variante du bootstrapping utilisée en TFHE permettant d'évaluer simultanément une fonction de look-up table arbitraire tout en réduisant le bruit.

PHE : Voir Chiffrement Homomorphique Partiel.

Post-quantique : Se dit d'un algorithme cryptographique supposé résistant aux attaques par ordinateur quantique (contrairement à RSA, ECC qui sont brisés par l'algorithme de Shor).

Ring-LWE : Variante du problème LWE opérant dans des anneaux polynomiaux, plus efficace grâce à la structure algébrique supplémentaire. Base des schémas BFV, BGV, CKKS.

SHE (Somewhat Homomorphic Encryption) : Schéma supportant les deux opérations (addition et multiplication) mais pour un nombre limité d'opérations avant que le bruit ne devienne trop important.

TFHE : Schéma FHE de Chillotti-Gama-Georgieva-Izabachène (2020), basé sur le tore, avec un bootstrapping ultra-rapide (< 26 ms par porte logique). Développé et optimisé par Zama.

Zero-trust : Paradigme de sécurité postulant qu'aucun acteur (interne ou externe) ne doit être considéré comme de confiance par défaut. Le FHE est un outil naturel de l'architecture zero-trust.

Annexe B — Tableau comparatif des schémas FHE

Critère	PHE (Paillier)	SHE (BGV/BFV)	CKKS	TFHE
Opérations	Addition seule	Addition + Mult.	Addition + Mult.	Toute fonction
Domaine	Entiers Z_n	Entiers Z_t	Réels approx.	Bits/petits entiers
Bootstrapping	Non requis	Possible (lent)	Oui (lent-moyen)	Oui (rapide, PBS)
Perf. addition	Rapide (ms)	Rapide (µs-ms)	Rapide (µs-ms)	Lente (ms)
Perf. mult.	N/A	Moyenne (ms)	Moyenne (ms)	Lente (ms/porte)
Ciphertext size	Petit (Ko)	Moyen (Ko)	Moyen (Ko)	Variable (Ko)
Post-quantique	Non (factoris.)	Oui (LWE)	Oui (LWE)	Oui (LWE)
Maturité	Haute	Haute	Haute	Haute
Bibliothèque réf.	Python-paillier	SEAL, OpenFHE	SEAL, HElib	Concrete, TFHE-rs
Usage principal	Vote, stats	DB chiffrée, AML	ML, génomique	Smart contracts, ML

Tableau B.1 — Comparatif détaillé des schémas PHE, SHE/BFV, CKKS et TFHE selon dix critères clés.

Annexe C — Exemple de code : addition homomorphique avec Concrete ML

L'exemple suivant illustre une addition homomorphique utilisant la bibliothèque Concrete de Zama. Ce code Python compile automatiquement un calcul simple vers le schéma TFHE :

```
# Installation : pip install concrete-python from concrete import fhe
@fhe.compiler({"x": "encrypted", "y": "encrypted"}) def add_encrypted(x, y):
return x + y  # Jeu d'entrainement pour déterminer les paramètres inputset = [(i,
j) for i in range(10) for j in range(10)]  # Compilation vers FHE circuit =
add_encrypted.compile(inputset)  # Chiffrement des entrées et calcul homomorphique
x_enc, y_enc = circuit.encrypt(3, 5) result_enc = circuit.run(x_enc, y_enc) result
= circuit.decrypt(result_enc)  print(f"Résultat déchiffré : {result}")  # Affiche :
8 # Le serveur n'a jamais eu accès à 3, 5, ou 8 en clair
```

Code C.1 — Addition homomorphique avec Concrete de Zama (source : docs.zama.ai).

Cet exemple illustre la philosophie de Concrete : le développeur écrit une fonction Python ordinaire (add_encrypted), la décore avec @fhe.compiler pour indiquer quels arguments doivent être chiffrés, puis fournit un jeu d'exemples (inputset) permettant au compilateur de déterminer automatiquement les paramètres cryptographiques optimaux. La transparence complète pour le développeur — qui n'a pas besoin de connaître LWE, Ring-LWE ou les paramètres du schéma TFHE — illustre la vision de démocratisation du FHE poursuivie par Zama.